Développement d'applications mobiles Android

Informatique — Licence 3 (L3)

Cours INF-L3-11

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Créer une application Android complète en Kotlin : architecture et composants, interfaces XML, navigation et cycle de vie des écrans, stockage local (préférences et SQLite), consommation d'une API REST en asynchrone, puis publication sur une boutique d'applications.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Architecture d'une application Android
• Leçon 2 — Interfaces mobiles : écrans et composants visuels
• Leçon 3 — Navigation et cycle de vie des écrans
• Leçon 4 — Stockage local : préférences partagées et base SQLite
• Leçon 5 — Consommer une API REST depuis l'application mobile
• Leçon 6 — Publication et bonnes pratiques des boutiques d'applications
• Leçon 7 — Projet : application mobile complète et fonctionnelle
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Le téléphone est devenu le premier écran du monde : messagerie, banque mobile, commerce, transport, études — presque tout passe désormais par des applications. Et dans ce paysage, Android équipe l'écrasante majorité des smartphones en circulation, en particulier en Afrique de l'Ouest, où le téléphone est souvent le seul ordinateur de la famille. Pour un étudiant en informatique, savoir construire une application Android complète n'est donc pas un luxe ni une spécialité exotique : c'est une compétence directement employable, de la start-up de Bamako qui numérise ses services à la banque ou à l'opérateur qui recrute des développeurs mobiles pour ses applications grand public.

Ce cours construit, étape par étape, une vraie application de bout en bout : « BiblioIAM Mobile », le compagnon de poche de la bibliothèque de l'institut. Elle permet de chercher un livre dans le catalogue en ligne, d'ouvrir son détail, de l'enregistrer sur le téléphone pour le retrouver hors ligne, et de préparer sa propre liste de lectures. Chaque leçon ajoute une brique indispensable à cet édifice — architecture et composants, interfaces, navigation et cycle de vie, stockage local, consommation d'une API REST, puis publication sur une boutique — et le projet final de la leçon 7 les assemble toutes dans une application démontrable. Le langage utilisé est Kotlin, le langage officiel recommandé par Google pour Android, avec des interfaces décrites en XML.

Le cours suppose que tu sais déjà programmer : programmation orientée objet rencontrée en L2 avec Java, bases de données et langage SQL, notions d'API REST et

format JSON déjà croisés en L3. Aucun téléphone haut de gamme n'est exigé pour suivre : Android Studio, l'environnement officiel et gratuit, fournit un émulateur qui simule un appareil sur ton ordinateur, et tous les exemples du cours sont courts, complets et vérifiables à la lecture avant même de les taper. À la fin du parcours, tu sauras expliquer chaque ligne d'une petite application réelle — et, surtout, tu connaîtras la marche complète pour la signer et la publier.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire l'architecture d'une application Android et le rôle de ses quatre composants fondamentaux.
- Construire une interface mobile en XML avec des composants visuels correctement reliés au code.
- Expliquer le cycle de vie d'une Activity dans l'ordre exact et gérer les changements d'état (rotation, arrière-plan).
- Stocker des données localement : préférences partagées pour les réglages, base SQLite avec des requêtes toujours paramétrées.
- Consommer une API REST en tâche asynchrone (coroutines) sans jamais bloquer le thread principal.
- Préparer une version signée (AAB) et décrire les étapes et bonnes pratiques de publication sur Google Play.
- Conduire un projet mobile complet, de l'écran d'accueil au scénario de démonstration final.

Leçon 1 — Architecture d'une application Android

Android est un système d'exploitation bâti sur un noyau Linux, surmonté d'un environnement d'exécution (ART) qui fait tourner le code compilé de chaque application dans son propre processus, isolé des autres. Point capital : une application Android n'a pas de fonction main() unique comme un programme de bureau. Elle est un assemblage de composants que le système démarre à la demande — quand l'utilisateur touche l'icône, quand une alarme sonne, quand un message arrive. C'est le système qui décide du cycle de vie de ces composants, et le développeur écrit des méthodes de rappel (callbacks) que le système appellera au bon moment.

Il existe exactement quatre types de composants fondamentaux, à ne pas confondre. L'Activity représente un écran avec une interface : c'est le composant visible et interactif (la liste des livres, l'écran de connexion). Le Service exécute une tâche de fond sans interface, comme lire de la musique ou télécharger un fichier pendant que l'utilisateur fait autre chose. Le BroadcastReceiver réagit à des annonces diffusées par le système ou par des applications — batterie faible, fin du démarrage, arrivée d'un SMS — en exécutant un traitement court. Le ContentProvider, enfin, sert à partager des données structurées avec d'autres applications, comme les contacts ou les photos, au moyen d'une interface de requêtes.

Tous ces composants sont déclarés dans un fichier central : AndroidManifest.xml, la carte d'identité de l'application. Il annonce le nom de package, les Activities (dont celle

qui sert de point d'entrée, avec le filtre MAIN / LAUNCHER), les permissions demandées (Internet, caméra…) et les versions du système visées. Le projet complet s'organise en trois familles : le code (Kotlin ou Java), les ressources du dossier res/ (mises en page XML, textes, images, couleurs) et les scripts de construction Gradle, qui compilent et assemblent le tout en un paquet installable. L'architecture moderne recommandée par Google sépare ensuite l'interface (UI), la logique de présentation (ViewModel) et les données (dépôts locaux et distants) : c'est le fil conducteur de ce cours.

L'essentiel
Une application Android = des composants (Activity, Service, BroadcastReceiver, ContentProvider) déclarés dans le manifeste, démarrés par le système, sans main() unique. Le code, les ressources res/ et la construction Gradle forment le projet.

Exemple — Le manifeste de BiblioIAM Mobile : point d'entrée et permissions

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

<!-- Permission : indispensable pour appeler l'API (leçon 5) -->
<uses-permission android:name="android.permission.INTERNET" />

<application
android:label="BiblioIAM Mobile"
android:theme="@style/Theme.BiblioIAM">

<!-- Écran d'accueil : le point d'entrée de l'application -->
<activity
android:name=".MainActivity"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>

<!-- Écran de détail d'un livre : pas d'accès direct -->
<activity
android:name=".DetailLivreActivity"
android:exported="false" />
</application>
</manifest>
```

À retenir

• Quatre composants, quatre rôles : Activity = écran, Service = tâche de fond, BroadcastReceiver = réaction aux annonces, ContentProvider = partage de données.

• Le manifeste déclare les composants, les permissions et l'Activity de démarrage (MAIN + LAUNCHER) ; une Activity non déclarée ne peut pas être lancée.

• android:exported vaut true seulement si un autre processus doit pouvoir lancer le composant ; l'écran d'accueil l'exige, les écrans internes non.

Exercice

Pour chacun de ces besoins, indique le composant adapté et justifie en une phrase : a) afficher l'écran de recherche de livres ; b) continuer un téléchargement de catalogue après la fermeture de l'écran ; c) relancer une synchronisation quand le téléphone retrouve du réseau et annonce « connexion rétablie » ; d) permettre à une autre application de lire, avec autorisation, la liste des emprunts. Puis explique ce qui se passerait si MainActivity n'était pas déclarée dans le manifeste.

✔ Corrigé
a) Activity : il y a un écran et une interaction. b) Service : tâche longue, sans interface, qui doit survivre à la fermeture de l'écran. c) BroadcastReceiver : il réagit à une annonce du système ; son traitement doit rester court (il peut ensuite démarrer un Service pour le vrai travail). d) ContentProvider : partage de données structurées entre applications, avec contrôle des accès. Si MainActivity n'est pas déclarée, l'application s'installe mais ne peut pas démarrer cet écran : le système ne connaît que les composants déclarés dans le manifeste, et une tentative de lancement se solderait par une erreur « Activity not found ».

Leçon 2 — Interfaces mobiles : écrans et composants visuels

L'interface d'un écran Android est un arbre de vues. Les feuilles sont les composants visuels (View) : TextView pour afficher du texte, EditText pour saisir, Button pour agir, ImageView pour les images, CheckBox, Switch et bien d'autres. Les nœuds sont les conteneurs (ViewGroup), qui disposent leurs enfants : LinearLayout les aligne en ligne ou en colonne, ConstraintLayout les positionne par contraintes relatives (au parent, aux voisins) — c'est le conteneur recommandé pour les écrans complexes — et FrameLayout les superpose. Cet arbre est le plus souvent décrit dans un fichier XML du dossier res/layout, ce qui sépare proprement la présentation du code Kotlin qui la pilote.

Chaque vue utile au code reçoit un identifiant unique, déclaré par @+id/nom dans le XML. Le code récupère alors la vue par findViewById(R.id.nom) — ou, mieux, par le mécanisme de View Binding qui génère des accès typés — puis il lui attache des comportements : un écouteur setOnClickListener sur un bouton, par exemple. Pour afficher de longues listes (des centaines de livres), on n'empile pas des TextView à la main : on utilise un RecyclerView, qui ne crée que les lignes visibles et recycle celles qui sortent de l'écran, guidé par un Adapter qui transforme les données en lignes. C'est le patron de toutes les listes mobiles.

Les tailles suivent des unités indépendantes de l'écran : le dp pour les dimensions (une taille tactile confortable vaut au moins 48 dp) et le sp pour le texte, qui respecte les préférences d'agrandissement de l'utilisateur. Les textes ne se codent jamais « en dur » dans le code : ils vivent dans res/values/strings.xml, ce qui prépare la traduction et la relecture. Les couleurs et les thèmes suivent le même principe, et Material Design — la charte graphique de Google — fournit des composants cohérents, des contrastes vérifiés et des règles d'accessibilité, comme un texte alternatif (contentDescription)

sur les images qui portent une information.

L'essentiel
Une interface = un arbre de vues décrit en XML : des View (TextView, EditText, Button…) dans des ViewGroup (LinearLayout, ConstraintLayout). Les id (@+id/…) relient le XML au code ; RecyclerView + Adapter affichent les longues listes ; textes et couleurs vivent dans les ressources.

Exemple — Écran de recherche : le layout XML et son pilotage en Kotlin (ids cohérents)

```
<!-- res/layout/activity_main.xml -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="16dp">
<EditText
android:id="@+id/champRecherche"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Titre ou auteur du livre" />
<Button
android:id="@+id/boutonRechercher"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Rechercher" />
<TextView
android:id="@+id/texteStatut"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Aucune recherche pour le moment" />
<androidx.recyclerview.widget.RecyclerView
android:id="@+id/listeLivres"
android:layout_width="match_parent"
android:layout_height="0dp"
android:layout_weight="1" />
</LinearLayout>
// MainActivity.kt : les id repris sont exactement ceux du XML ci-dessus
class MainActivity : AppCompatActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)

val champ = findViewById<EditText>(R.id.champRecherche)
val statut = findViewById<TextView>(R.id.texteStatut)

findViewById<Button>(R.id.boutonRechercher).setOnClickListener {
val mot = champ.text.toString().trim()
statut.text = if (mot.isEmpty())
"Tape d'abord un titre ou un auteur"
else
"Recherche de : $mot"
}
}
}
```

À retenir

• match_parent = prendre toute la place du parent ; wrap_content = juste la place du contenu ; 0dp + poids 1 dans un LinearLayout = occuper l'espace restant.

• Un id déclaré dans le XML (@+id/champRecherche) se retrouve dans le code sous la forme R.id.champRecherche : toute faute de frappe ou d'orthographe casse la liaison.

• RecyclerView ne crée que les lignes visibles et les recycle : c'est l'outil obligatoire dès qu'une liste peut grandir.

Exercice

Dans le layout de l'exemple, l'étudiant doit ajouter, sous le champ de recherche, un second EditText d'id champAuteur permettant de filtrer aussi par auteur, et un bouton d'id boutonEffacer. Écris le fragment XML correspondant (avec des hint parlants), puis la ligne Kotlin qui récupère le champ auteur. Explique enfin pourquoi la RecyclerView utilise layout_height="0dp" avec un poids, plutôt qu'une hauteur fixe en dp.

✔ Corrigé
XML à insérer après le premier EditText : <EditText android:id="@+id/champAuteur" android:layout_width="match_parent" android:layout_height="wrap_content" android:hint="Auteur (optionnel)" /> puis <Button android:id="@+id/boutonEffacer" ... android:text="Effacer" />. En Kotlin : val champAuteur = findViewById<EditText>(R.id.champAuteur). La hauteur 0dp + android:layout_weight="1" signifie : dans le LinearLayout vertical, la liste prend tout l'espace qui reste après les champs et boutons, quel que soit l'écran. Une hauteur fixe (par exemple 400dp) déborderait sur un petit téléphone et laisserait un vide disgracieux sur une tablette.

Leçon 3 — Navigation et cycle de vie des écrans

Passer d'un écran à l'autre se fait avec un Intent, un message qui demande au système d'ouvrir un composant. L'Intent explicite nomme la destination dans l'application — de MainActivity vers DetailLivreActivity — et transporte des données dans ses « extras » : des paires clé-valeur, comme l'identifiant du livre à afficher. L'Intent implicite, lui, décrit une action générique (ouvrir une page web, composer un numéro, partager un texte) et laisse le système proposer les applications capables de la traiter. Les écrans ouverts s'empilent dans une pile (la back stack) : le bouton Retour dépile l'écran courant et redonne la main au précédent, exactement comme on reviendrait en arrière dans un navigateur.

Chaque Activity traverse un cycle de vie strict, que le système pilote en appelant des méthodes de rappel dans un ordre immuable. À l'ouverture : onCreate (créer l'écran, charger le layout, initialiser), puis onStart (l'écran devient visible), puis onResume (l'écran est au premier plan et interactif). Quand l'utilisateur part ailleurs ou qu'un écran transparent la recouvre : onPause (elle perd le premier plan — rester bref ici). Quand elle n'est plus visible du tout : onStop. Enfin, onDestroy marque la destruction définitive. Si l'utilisateur revient sur une Activity arrêtée mais non détruite, le système passe par onRestart, puis onStart et onResume. Retenir la chaîne complète — onCreate → onStart → onResume → onPause → onStop → onDestroy — est LA question classique de ce chapitre.

Ce cycle n'est pas une curiosité théorique : il gouverne des comportements réels. Tourner le téléphone (changement de configuration) détruit et recrée l'Activity visible : sans précaution, le texte tapé et la position de défilement disparaissent — on les conserve via onSaveInstanceState ou, proprement, dans un ViewModel qui survit à la recréation. Un appel entrant envoie l'écran en onPause puis onStop : c'est donc dans onPause qu'on met en veille ce qui doit s'arrêter tout de suite (aperçu caméra, enregistrement), et dans onResume qu'on le relance. Enfin, quand le système manque

de mémoire, il peut tuer le processus d'une application en arrière-plan sans prévenir : une application bien conçue sauvegarde tôt et souvent, au lieu de supposer qu'onDestroy sera toujours appelé — il ne l'est pas garanti.

L'essentiel
Navigation = Intent (explicite avec extras, ou implicite) et pile d'écrans. Cycle de vie, dans l'ordre : onCreate → onStart → onResume → (onPause → onStop) → onDestroy, avec onRestart au retour. Rotation = destruction puis recréation : l'état se sauvegarde, il ne se suppose pas.

Exemple — Schéma du cycle de vie et journalisation dans l'écran de détail

```
CYCLE DE VIE D'UNE ACTIVITY (chemin normal)

[Créée] --onCreate--> [Visible] --onStart--> [Premier plan]
|
onResume
v
[Détruite] <--onDestroy-- [Arrêtée] <--onStop-- [En pause]
| ^
+--onRestart--> onStart --> onResume
(retour de l'utilisateur)

Règles : onPause précède TOUJOURS onStop ; onDestroy n'est
PAS garanti si le système tue le processus en arrière-plan.

// DetailLivreActivity.kt : tracer le cycle dans Logcat
class DetailLivreActivity : AppCompatActivity() {

override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_detail_livre)
Log.d("BiblioIAM", "onCreate : écran créé")

// L'id du livre arrive par l'Intent de MainActivity
val idLivre = intent.getLongExtra("EXTRA_ID_LIVRE", -1L)
val titre = findViewById<TextView>(R.id.texteTitreDetail)
titre.text = "Détail du livre n° $idLivre"
}

override fun onStart() { super.onStart()
Log.d("BiblioIAM", "onStart : écran visible") }

override fun onResume() { super.onResume()
Log.d("BiblioIAM", "onResume : premier plan") }

override fun onPause() { super.onPause()
Log.d("BiblioIAM", "onPause : perte du premier plan") }

override fun onStop() { super.onStop()
Log.d("BiblioIAM", "onStop : écran invisible") }

override fun onDestroy() { super.onDestroy()
Log.d("BiblioIAM", "onDestroy : écran détruit") }
}
```

À retenir

• L'ordre est fixe : onCreate, onStart, onResume à l'arrivée ; onPause, onStop, onDestroy au départ — onPause ne vient jamais après onStop.

• Les extras d'un Intent voyagent avec des clés constantes ("EXTRA_ID_LIVRE") et une valeur par défaut, au cas où la clé serait absente.

• La rotation détruit et recrée l'écran : ViewModel ou onSaveInstanceState conservent l'état ; onDestroy n'est pas garanti en cas de manque de mémoire.

Exercice

Scénario : Awa ouvre BiblioIAM Mobile, tape sur un livre (l'écran de détail s'affiche), reçoit un appel téléphonique, y répond pendant deux minutes, raccroche, revient sur le détail, puis appuie deux fois sur Retour jusqu'à quitter l'application. Liste, dans l'ordre, toutes les méthodes de rappel traversées par DetailLivreActivity, et précise à quel moment le système a pu la détruire sans appeler onDestroy.

✔ Corrigé
Ouverture du détail : onCreate → onStart → onResume. L'appel entrant met l'application au second plan : onPause → onStop. Retour après l'appel : onRestart → onStart → onResume. Premier Retour : le détail est dépilé et détruit : onPause → onStop → onDestroy. (Le second Retour quitte MainActivity, qui traverse la même fin de chaîne.) Le moment de destruction silencieuse : pendant l'appel, quand l'écran est en état « arrêté » (après onStop), le système peut tuer le processus entier pour libérer de la mémoire, sans appeler onDestroy — d'où la règle : tout ce qui compte doit déjà être sauvegardé à ce stade.

Leçon 4 — Stockage local : préférences partagées et base SQLite

Une application sérieuse ne redemande pas tout à chaque lancement : elle conserve des données sur l'appareil, dans un espace privé — le « bac à sable » — inaccessible aux autres applications. Trois niveaux de stockage répondent à trois besoins. Les préférences partagées (SharedPreferences) stockent des paires clé-valeur simples : le nom de l'utilisateur, le thème sombre ou clair, le dernier écran visité. Les fichiers internes conviennent aux contenus volumineux ou spécifiques (un catalogue exporté en JSON, une image). Et pour des données structurées, nombreuses et interrogeables — les livres, les emprunts —, il faut une vraie base de données : SQLite, intégrée à Android, qui fonctionne sans serveur et survit à la fermeture de l'application.

L'accès à SQLite se fait par une classe d'aide qui hérite de SQLiteOpenHelper. Elle déclare le nom de la base et un numéro de version de schéma ; le système l'appelle une seule fois, à la première utilisation, via onCreate, où l'on crée les tables avec CREATE TABLE, puis via onUpgrade quand la version augmente et que le schéma doit évoluer. Les écritures passent par des ContentValues, dictionnaire colonne-valeur qui évite de composer le SQL à la main, et les lectures par un Cursor que l'on parcourt ligne à ligne. Règle de sécurité absolue de ce cours : une requête reçoit ses valeurs par des paramètres liés (le point d'interrogation « ? » et un tableau d'arguments), JAMAIS par concaténation de chaînes — sinon, le premier guillemet tapé par un utilisateur casse la requête, et un texte malveillant peut devenir une injection SQL.

Au-dessus de SQLite, Google recommande aujourd'hui la bibliothèque Room : les tables deviennent des classes annotées, les requêtes sont vérifiées à la compilation et le code répétitif disparaît. Il faut néanmoins comprendre SQLite « à la main » d'abord — c'est ce que fait ce cours — car Room ne fait que générer ce même travail. Deux disciplines complètent le tableau : fermer les curseurs après usage (ou utiliser use {} qui le fait automatiquement), et ne pas lancer de grosses requêtes sur le thread principal, sous peine de figer l'interface ; les accès importants se font en tâche de fond, par exemple dans une coroutine.

L'essentiel
SharedPreferences pour les petits réglages clé-valeur ; SQLite pour les données structurées, via SQLiteOpenHelper (onCreate crée les tables, onUpgrade fait évoluer le schéma). Requêtes TOUJOURS paramétrées (? + arguments), curseurs fermés, Room comme couche moderne au-dessus.

Exemple — BiblioDbHelper : création de la table, insertion et recherche paramétrée

```
class BiblioDbHelper(context: Context) :
SQLiteOpenHelper(context, "biblioiam.db", null, 1) {

// Appelée une seule fois : à la création de la base
override fun onCreate(db: SQLiteDatabase) {
db.execSQL(
"CREATE TABLE livres (" +
"id INTEGER PRIMARY KEY AUTOINCREMENT, " +
"titre TEXT NOT NULL, auteur TEXT NOT NULL, " +
"annee INTEGER, " +
"disponible INTEGER NOT NULL DEFAULT 1)"
)
}

// Version en hausse : repartir de zéro (exercice seulement)
override fun onUpgrade(db: SQLiteDatabase, anc: Int, nouv: Int) {
db.execSQL("DROP TABLE IF EXISTS livres")
onCreate(db)
}

fun ajouterLivre(titre: String, auteur: String, annee: Int): Long {
val valeurs = ContentValues().apply {
put("titre", titre); put("auteur", auteur)
put("annee", annee); put("disponible", 1)
}
return writableDatabase.insert("livres", null, valeurs)
}

// Recherche SÛRE : le mot est un paramètre lié, jamais
// concaténé : il ne peut ni casser ni injecter la requête.
fun rechercherLivres(mot: String): List<String> {
val curseur = readableDatabase.query(
"livres", arrayOf("titre", "auteur"),
"titre LIKE ?", arrayOf("%$mot%"),
null, null, "titre ASC"
)
val resultat = mutableListOf<String>()
curseur.use {
while (it.moveToNext()) {
resultat.add(it.getString(0) + " — " + it.getString(1))
}
}
return resultat
}
}
```

À retenir

• SQLiteOpenHelper : onCreate n'est appelé qu'à la première utilisation de la base ; changer le numéro de version déclenche onUpgrade.

• INSERT sûr = ContentValues ; SELECT sûr = clause "colonne LIKE ?" + tableau d'arguments : la concaténation d'une saisie utilisateur dans du SQL est interdite, sans exception.

• Un Cursor se ferme toujours (use {} le fait pour nous) ; SharedPreferences suffit pour un réglage, pas pour 500 livres.

Exercice

1) Écris la méthode supprimerLivre(id: Long): Int de BiblioDbHelper, en respectant la règle de paramétrage, qui supprime le livre d'identifiant donné et renvoie le nombre de lignes supprimées. 2) Un camarade propose : db.execSQL("DELETE FROM livres WHERE id = " + idSaisi) où idSaisi vient d'un champ de texte. Explique le danger concret avec un exemple de saisie. 3) La version 2 de l'application ajoute une colonne « rayon » à la table livres : écris un onUpgrade qui conserve les données existantes.

✔ Corrigé

1) fun supprimerLivre(id: Long): Int = writableDatabase.delete("livres", "id = ?", arrayOf(id.toString())) — delete renvoie le nombre de lignes affectées. 2) Avec la concaténation, la saisie cesse d'être une donnée et devient du SQL : taper « 1 OR 1=1 » transformerait la suppression en « DELETE FROM livres WHERE id = 1 OR 1=1 », vraie pour toutes les lignes : la table entière est effacée. Avec un paramètre, la même saisie est comparée comme une simple chaîne à la colonne id et ne correspond à rien. 3) onUpgrade conserve les données : if (anc < 2) { db.execSQL("ALTER TABLE livres ADD COLUMN rayon TEXT") } — on ajoute la colonne sans détruire la table ; supprimer puis recréer la table (comme dans l'exemple du cours) n'est acceptable qu'en exercice, car en production il effacerait les livres de tous les utilisateurs.

Leçon 5 — Consommer une API REST depuis l'application mobile

L'application ne peut pas embarquer toutes les données du monde : le catalogue complet de la bibliothèque, ses disponibilités en temps réel, vivent sur un serveur. L'application devient cliente d'une API REST : elle envoie des requêtes HTTP — GET pour lire, POST pour créer, PUT pour modifier, DELETE pour supprimer — et reçoit des réponses en JSON accompagnées d'un code de statut (200 succès, 201 créé, 404 introuvable, 500 erreur du serveur). Chaque objet JSON reçu doit être converti en objet du langage : une classe de données (data class) Kotlin dont les champs reflètent le JSON, convertie automatiquement par une bibliothèque comme Gson ou Moshi.

La règle d'or du réseau mobile : jamais d'appel réseau sur le thread principal, celui qui dessine l'interface et réagit aux doigts de l'utilisateur. Android le sanctionne d'ailleurs par une exception NetworkOnMainThreadException qui fait planter l'application. En Kotlin, on délègue l'appel à une coroutine : une fonction « suspend » décrivant un travail qui peut attendre sans bloquer, lancée dans un périmètre lié à l'écran

(lifecycleScope) et basculée sur le répartiteur d'entrées-sorties (Dispatchers.IO). La bibliothèque Retrofit simplifie encore la vie : on déclare chaque route de l'API par une interface annotée (@GET, @POST…), et Retrofit génère le code d'appel, la conversion JSON incluse. Sans oublier le préalable : la permission INTERNET déclarée dans le manifeste, sinon chaque requête échoue avant même de partir.

Une bonne cliente gère enfin la vraie vie : l'écran de chargement pendant l'attente, le message clair quand le serveur répond une erreur, le bouton « réessayer » quand le réseau manque — fréquent sur mobile — et une copie locale (la base SQLite de la leçon 4) pour continuer à travailler hors ligne avec les derniers livres consultés. Côté sécurité : toujours HTTPS, et si l'API exige une identification, le jeton d'accès voyage en en-tête Authorization ; en revanche, aucun secret durable — clé privée, mot de passe de service — ne doit être enfoui dans l'application distribuée, car un APK se décompile et livrerait le secret au premier curieux.

L'essentiel
API REST = requêtes HTTP + réponses JSON converties en data class. Appel TOUJOURS asynchrone (coroutine, Dispatchers.IO) : le thread principal ne touche jamais au réseau. Retrofit transforme une interface annotée en client ; permission INTERNET obligatoire ; prévoir chargement, erreurs et hors-ligne.

Exemple — Client Retrofit de BiblioIAM : modèle, interface et appel en coroutine

```
// 1) La data class reflète le JSON renvoyé par l'API
data class LivreDto(
val id: Long,
val titre: String,
val auteur: String,
val disponible: Boolean
)

// 2) L'interface décrit les routes de l'API (annotations)
interface BiblioApi {
@GET("livres")
suspend fun livres(@Query("q") recherche: String): List<LivreDto>

@GET("livres/{id}")
suspend fun livre(@Path("id") id: Long): LivreDto
}

// 3) Le client Retrofit, construit une seule fois
object BiblioClient {
val api: BiblioApi = Retrofit.Builder()
.baseUrl("https://api.biblio-iam.example/")
.addConverterFactory(GsonConverterFactory.create())
.build()
.create(BiblioApi::class.java)
}

// 4) L'appel, depuis MainActivity : coroutine liée à l'écran,
// bascule explicite sur le répartiteur d'entrées-sorties
private fun lancerRecherche(mot: String) {
lifecycleScope.launch {
texteStatut.text = "Chargement…"
try {
val livres = withContext(Dispatchers.IO) {
BiblioClient.api.livres(mot)
}
texteStatut.text = "${livres.size} livre(s) trouvé(s)"
adaptateur.soumettre(livres) // mise à jour de la liste
} catch (e: Exception) {
texteStatut.text = "Réseau indisponible : réessaie"
}
}
}
```

À retenir

• suspend + lifecycleScope.launch : la coroutine attend la réponse sans bloquer l'écran, et elle est annulée proprement quand l'écran disparaît.

• Sans <uses-permission INTERNET> dans le manifeste, aucun appel ne part ; avec un appel sur le thread principal, l'application plante (NetworkOnMainThreadException).

• Le JSON et la data class doivent correspondre champ par champ : un nom différent exige une annotation de correspondance (@SerializedName avec Gson).

Exercice

L'API de BiblioIAM renvoie pour un emprunt : {"id": 7, "livreId": 3, "membre": "Awa", "dateRetour": "2026-06-15", "rendu": false}. 1) Écris la data class Kotlin correspondante. 2) Ajoute à l'interface BiblioApi la route qui liste les emprunts d'un membre : GET emprunts?membre=<nom>. 3) Un camarade écrit l'appel directement dans onCreate, sans coroutine : que se passe-t-il au lancement, et pourquoi ?

✔ Corrigé
1) data class EmpruntDto(val id: Long, val livreId: Long, val membre: String, val dateRetour: String, val rendu: Boolean). 2) @GET("emprunts") suspend fun emprunts(@Query("membre") membre: String): List<EmpruntDto>. 3) L'appel réseau s'exécuterait sur le thread principal : Android l'interdit et lève NetworkOnMainThreadException — l'application plante dès l'ouverture de l'écran (ou, si elle ne plantait pas, l'interface gèlerait pendant toute la durée de la requête). L'appel doit vivre dans une coroutine lancée hors du thread d'interface, comme dans l'exemple du cours.

Leçon 6 — Publication et bonnes pratiques des boutiques d'applications

Publier commence par produire une version dite « release », différente de la version de débogage utilisée pendant le développement : code optimisé et minifié, journaux retirés, et surtout signature cryptographique obligatoire. L'application est signée avec un certificat conservé dans un fichier coffre (le keystore) protégé par mots de passe. Ce certificat est l'identité définitive de l'application : Google Play n'accepte une mise à jour que si elle porte exactement la même signature que la version initiale. Perdre le keystore, c'est donc perdre à jamais la possibilité de mettre à jour son application — il faudrait la republier sous un nouveau nom de package, en repartant de zéro, avis et utilisateurs perdus. On le sauvegarde donc en plusieurs endroits sûrs, hors du dépôt de code public.

Chaque version publiée porte deux numéros : versionCode, un entier strictement croissant que la boutique compare pour décider qu'une version est plus récente, et versionName, le nom lisible affiché aux utilisateurs (« 1.2.0 »). Le format de distribution attendu par Google Play est l'AAB (Android App Bundle) : on envoie un paquet complet, et Google génère pour chaque appareil l'APK optimisé à sa taille d'écran, sa langue et son architecture. Côté console, la publication exige un compte développeur (inscription unique payante), une fiche produit soignée — titre, descriptions courte et complète, icône, captures d'écran —, la classification du contenu, la tranche d'âge visée, et une politique de confidentialité dès que l'application collecte des données personnelles.

La publication n'est pas une ligne d'arrivée mais un début d'exploitation. Les bonnes pratiques : tester d'abord en pistes fermées (test interne, puis test fermé avec de vrais utilisateurs) avant la production, déployer progressivement pour limiter l'impact d'un bogue, et surveiller les indicateurs de qualité de la console (plantages, lenteurs — les « Android vitals ») car une application instable est rétrogradée. Il faut aussi répondre aux avis, justifier chaque permission demandée (une permission inutile effraie les utilisateurs et peut motiver un refus), et respecter les règles de la boutique : pas de

contenu trompeur, pas de collecte cachée, des mises à jour qui n'ajoutent pas de fonctions surprises. Une application honnête, sobre en permissions et régulièrement entretenue inspire confiance — et dure.

L'essentiel
Release = code optimisé + signature par keystore (à sauvegarder comme un trésor : sa perte interdit toute mise à jour). versionCode croissant obligatoire, versionName pour les humains, format AAB pour Google Play. Fiche complète, tests en pistes fermées, déploiement progressif et permissions justifiées.

Exemple — Configuration de signature et de version dans build.gradle.kts

```
android {
namespace = "ml.iam.biblioiam"
compileSdk = 34

defaultConfig {
applicationId = "ml.iam.biblioiam" // identité définitive
minSdk = 24
targetSdk = 34
versionCode = 3 // +1 à CHAQUE publication
versionName = "1.2.0" // nom affiché aux utilisateurs
}

signingConfigs {
create("release") {
// Le keystore et ses mots de passe ne sont JAMAIS
// écrits en clair ni poussés dans le dépôt Git :
// ils sont lus depuis un fichier local non suivi.
storeFile = file("biblioiam-release.jks")
storePassword = System.getenv("KEYSTORE_PASSWORD")
keyAlias = "biblioiam"
keyPassword = System.getenv("KEY_PASSWORD")
}
}

buildTypes {
getByName("release") {
isMinifyEnabled = true // code optimisé et réduit
signingConfig = signingConfigs.getByName("release")
}
}
}

// Générer le paquet à publier : Build > Generate Signed
// Bundle/APK > Android App Bundle (.aab) > release
```

À retenir

- Même signature ou rien : une mise à jour signée avec une autre clé est refusée ; le keystore se sauvegarde hors du code et en plusieurs exemplaires.

• versionCode doit augmenter à chaque envoi, sinon la console rejette le paquet ; versionName ne sert qu'à l'affichage.

• L'AAB laisse Google fabriquer l'APK adapté à chaque appareil : c'est le format de publication standard sur Google Play.

Exercice

Trois incidents de publication : a) l'équipe envoie une version avec le même versionCode que la précédente ; b) le seul exemplaire du keystore était sur un disque dur qui vient de lâcher, et une mise à jour urgente doit sortir ; c) la console signale un risque de refus car l'application demande l'accès aux SMS alors qu'elle ne sert qu'à consulter un catalogue. Pour chaque incident : conséquence exacte et conduite correcte (ou prévention, quand il est trop tard).

✔ Corrigé

a) Rejet immédiat par la console : un versionCode déjà utilisé ne peut pas être réutilisé ; on incrémente (3 → 4) et on renvoie. b) Catastrophe : sans le keystore d'origine, aucune mise à jour signée à l'identique n'est possible ; pour une application inscrite à la signature par Google Play, on peut demander une réinitialisation de clé d'envoi à Google (procédure encadrée, à identité vérifiée) — sinon, il faut republier sous un nouvel applicationId, en perdant les utilisateurs existants. Prévention : copies du keystore chiffrées, en lieux distincts, dès le premier jour. c) Retirer la permission inutile du manifeste : chaque permission doit correspondre à une fonction réelle et être justifiable dans la fiche ; une permission sensible injustifiée expose à un refus de la boutique et détruit la confiance des utilisateurs.

Leçon 7 — Projet : application mobile complète et fonctionnelle

Ce projet assemble toutes les briques du cours en une application livrable : BiblioIAM Mobile, le compagnon de poche de la bibliothèque de l'IAM. Le cahier des charges de la version 1 tient en quatre écrans : l'accueil affiche le champ de recherche et la liste des livres ; l'écran de détail présente un livre et son état de disponibilité ; l'écran « Ma bibliothèque » conserve, hors ligne, les livres enregistrés par l'étudiant ; un écran « À propos » rappelle les horaires de la bibliothèque. Les fonctions sont volontairement bornées : chercher en ligne, consulter le détail, enregistrer localement, retrouver ses livres sans réseau. Tout le reste — réservation réelle, paiement des pénalités — est explicitement hors périmètre de la version 1, et le dire par écrit fait partie du travail d'ingénieur.

L'organisation du code suit l'architecture vue en leçon 1, en paquets clairs : ui (les Activities et l'Adapter de la liste), data/local (BiblioDbHelper et les préférences), data/remote (l'interface Retrofit et le client), model (les classes Livre et LivreDto). Le flux de données reste simple et racontable : l'Activity demande, la couche remote interroge l'API dans une coroutine, les résultats s'affichent, et tout livre consulté est mémorisé dans SQLite pour alimenter « Ma bibliothèque ». Cette séparation paie immédiatement : changer d'adresse d'API ne touche qu'un fichier ; tester la base ne réclame pas d'écran.

Le travail se conduit en jalons courts, avec une version qui fonctionne à chaque étape : d'abord les écrans et la navigation avec des données fictives, ensuite le stockage local, ensuite l'API réelle, enfin la préparation de la publication (icône, version signée, fiche de test interne). La recette finale est une démonstration scénarisée sur appareil réel ou émulateur : recherche en ligne, mode avion activé pour prouver le hors-ligne, rotation de l'écran pour prouver que l'état survit, base vide pour prouver que l'application ne plante pas sur un cas limite. Une application qui passe ces quatre épreuves, avec un code rangé et expliqué, est un projet de L3 réussi — et une pièce réelle à montrer dans un portfolio.

L'essentiel
Projet BiblioIAM Mobile : 4 écrans, recherche en ligne via l'API, mémorisation locale SQLite, consultation hors ligne. Paquets ui / data/local / data/remote / model, jalons avec version fonctionnelle à chaque étape, recette scénarisée (en ligne, mode avion, rotation, base vide).

Exemple — Arborescence du projet et correspondance écrans — composants — données

```
app/src/main/
├── AndroidManifest.xml (leçon 1 : composants + INTERNET)
├── java/ml/iam/biblioiam/
│ ├── ui/
│ │ ├── MainActivity.kt (accueil + recherche)
│ │ ├── DetailLivreActivity.kt (détail d'un livre)
│ │ ├── MaBibliothequeActivity.kt(livres hors ligne)
│ │ └── LivreAdapter.kt (lignes du RecyclerView)
│ ├── data/
│ │ ├── local/
│ │ │ └── BiblioDbHelper.kt (leçon 4 : SQLite)
│ │ └── remote/
│ │ └── BiblioApi.kt (leçon 5 : Retrofit)
│ └── model/
│ └── Livre.kt (data class métier)
└── res/
├── layout/ (un XML par écran, leçon 2)
├── values/ (strings.xml, couleurs, thèmes)
└── drawable/ (icônes et images)

ÉCRAN SOURCE DE DONNÉES LECON
Accueil API (GET livres?q=) 5
Détail API (GET livres/{id}) 5
Ma bibliothèque SQLite local (hors ligne) 4
À propos strings.xml (statique) 2
```

À retenir

• Un périmètre écrit, avec ce qui est EXCLU de la version 1, protège le projet des demandes infinies et rend la démonstration honnête.

• Chaque écran du projet renvoie à une leçon du cours : le projet est la synthèse, pas un sujet nouveau.

• La recette se joue sur des cas limites (hors-ligne, rotation, base vide), pas seulement sur le chemin heureux.

Exercice

Rédige la grille de recette du projet : cinq cas de test précis, chacun avec l'action de l'utilisateur, le résultat attendu et la leçon du cours qu'il valide. Au moins un cas doit couvrir le hors-ligne et un cas la rotation de l'écran. Ajoute la phrase exacte qui, dans le rapport de projet, définit le périmètre de la version 1.

✔ Corrigé

Exemple de grille valide : 1) Rechercher « réseaux » avec le Wi-Fi actif → la liste affiche les livres correspondants de l'API et le statut annonce leur nombre (leçon 5). 2) Ouvrir un détail, activer le mode avion, ouvrir « Ma bibliothèque » → le livre consulté reste affiché depuis SQLite, sans plantage (leçon 4). 3) Taper une recherche, tourner l'écran → le texte saisi et les résultats sont conservés (leçon 3, ViewModel ou état sauvegardé). 4) Vider la base puis ouvrir « Ma bibliothèque » → un message « Aucun livre enregistré » s'affiche, aucun plantage (leçons 2 et 4). 5) Rechercher un titre inexistant → l'API renvoie une liste vide et l'écran l'explique calmement, sans erreur technique affichée (leçon 5). Phrase de périmètre : « La version 1 de BiblioIAM Mobile permet de consulter le catalogue et de conserver des livres hors ligne ; elle ne gère ni la réservation ni le paiement, réservés à une étude ultérieure. »

Travaux dirigés (TD)

TD 1 — Déboguer une application qui plante

Trois étudiants rapportent trois plantages de leur BiblioIAM Mobile. Cas A : l'application se ferme dès qu'on appuie sur « Rechercher » ; dans le code, l'appel à l'API est fait directement dans le gestionnaire de clic, sans coroutine, et le journal annonce NetworkOnMainThreadException. Cas B : la recherche par auteur fonctionne pour « Hugo » mais plante avec « L'Harmattan d'Or » ; le code construit la requête ainsi : "SELECT * FROM livres WHERE auteur = '" + auteur + "'". Cas C : après une rotation de l'écran, la liste des résultats disparaît et la recherche repart de zéro ; aucun ViewModel n'est utilisé et rien n'est sauvegardé. Pour chaque cas : cause précise, correction, et règle du cours violée.

✔ Corrigé du TD 1
Cas A — Cause : un appel réseau sur le thread principal, interdit par Android, d'où l'exception. Correction : envelopper l'appel dans lifecycleScope.launch et withContext(Dispatchers.IO), comme en leçon 5. Règle violée : le thread d'interface ne fait ni réseau ni calcul long. Cas B — Cause : la requête est construite par concaténation ; l'apostrophe de « L'Harmattan » ferme prématurément la chaîne SQL et casse la syntaxe (et un auteur malveillant pourrait injecter du SQL). Correction : requête paramétrée, "auteur = ?" avec l'auteur passé en argument : l'apostrophe redevient une simple donnée. Règle violée : leçon 4, paramétrage obligatoire. Cas C — Cause : la rotation détruit et recrée l'Activity ; les résultats, gardés dans de simples variables d'écran, sont perdus. Correction : conserver les résultats dans un ViewModel (qui survit à la recréation) ou sauvegarder l'état dans onSaveInstanceState. Règle violée : leçon 3, l'état ne se suppose pas, il se sauvegarde.

TD 2 — Choisir le bon stockage pour chaque donnée

Une application d'emploi du temps pour étudiants de l'IAM doit conserver : a) le choix du thème (clair/sombre) et le nom de l'utilisateur ; b) 400 cours planifiés (matière, enseignant, salle, jour, heure) à filtrer par jour et par matière ; c) la photo de profil de l'utilisateur ; d) le jeton de session temporaire pour l'API ; e) l'historique des 10 dernières recherches. Pour chaque donnée : mécanisme choisi (SharedPreferences, fichier interne, SQLite), emplacement dans l'architecture, et justification en une phrase. Pour la donnée (b), écris le CREATE TABLE adapté.

✔ Corrigé du TD 2
a) SharedPreferences : deux paires clé-valeur lues au démarrage, rien à interroger. b) SQLite : données structurées, nombreuses, filtrées par critères — une requête paramétrée par jour et par matière. CREATE TABLE cours (id INTEGER PRIMARY KEY AUTOINCREMENT, matiere TEXT NOT NULL, enseignant TEXT, salle TEXT, jour TEXT NOT NULL, heure_debut TEXT NOT NULL). c) Fichier interne (dossier de fichiers de l'application) : une image est un contenu binaire volumineux, pas une ligne de base ni une préférence. d) SharedPreferences privées au minimum — et idéalement le stockage chiffré du système ; c'est une donnée courte, relue à chaque appel d'API, jamais affichée. e) SQLite ou SharedPreferences selon le besoin : si l'on veut seulement les 10 dernières recherches affichées sans recherche dedans, une petite table SQLite (mot, date) avec suppression au-delà de 10 est propre ; les SharedPreferences suffiraient pour une simple liste fixe. Critère général du TD : clé-valeur simple → préférences ; contenu binaire → fichier ; données structurées interrogeables → SQLite.

Travaux pratiques (TP)

TP 1 — Réaliser BiblioIAM Mobile minimal, de l'écran vide à la version signée

Durée indicative : 4 séances. Étape 1 — Projet et écrans : crée le projet Kotlin dans Android Studio, déclare les composants dans le manifeste, construis le layout de recherche de la leçon 2 et la navigation vers l'écran de détail avec passage de l'identifiant par Intent (leçon 3) ; les données sont fictives à ce stade. Étape 2 —

Stockage : implémente BiblioDbHelper (leçon 4), un bouton « Enregistrer » sur le détail et l'écran « Ma bibliothèque » hors ligne. Étape 3 — Réseau : branche l'API de démonstration avec Retrofit (leçon 5) : états chargement / résultats / erreur, mode avion testé. Étape 4 — Finition : icône, strings relus, versionCode 1 et versionName « 1.0.0 », génération d'un bundle release signé avec un keystore créé pour l'exercice, test en piste interne simulée (installation sur un second appareil ou émulateur propre). Livrables : le projet complet, un fichier LISEZMOI décrivant l'architecture en 15 lignes, et la grille de recette de la leçon 7 remplie.

✔ Attendus du TP 1
Attendus : le projet compile et s'installe ; la recherche en ligne affiche les résultats de l'API dans le RecyclerView ; le détail reçoit le bon identifiant (vérifier avec deux livres différents) ; un livre enregistré se retrouve dans « Ma bibliothèque » après fermeture complète de l'application et en mode avion ; la rotation conserve l'état de la recherche ; le bundle .aab release est généré et signé, et le keystore est conservé HORS du dossier du projet versionné. Barème indicatif : écrans et navigation 25 %, stockage local sûr (requêtes paramétrées visibles dans le code) 25 %, réseau asynchrone et gestion d'erreurs 25 %, finition et recette documentée 25 %. Erreurs qui font perdre la moitié des points d'une partie : appel réseau sur le thread principal, concaténation SQL, keystore poussé dans Git.

Exercices corrigés supplémentaires

Exercice 1

Scénario de callbacks : Fatou ouvre l'application (écran d'accueil), ouvre le détail d'un livre, appuie sur le bouton Accueil du téléphone pour répondre à un message, revient à l'application par l'icône, puis appuie sur Retour. Donne la suite exacte des méthodes appelées sur DetailLivreActivity, sans en oublier aucune, et indique l'état de l'écran après chaque groupe.

✔ Corrigé
Ouverture du détail : onCreate, onStart, onResume (écran au premier plan). Bouton Accueil : onPause, onStop (écran invisible, processus encore vivant en principe). Retour par l'icône : onRestart, onStart, onResume (l'écran n'avait pas été détruit : pas de nouvel onCreate). Bouton Retour : onPause, onStop, onDestroy (l'écran est dépilé de la pile et détruit définitivement). Piège évité : confondre le retour après Accueil (onRestart) avec une création nouvelle — onCreate n'apparaît qu'une seule fois dans tout le scénario.

Exercice 2

L'API renvoie : [{"id": 12, "titre": "Réseaux informatiques", "auteur": "A. Tanenbaum", "nbExemplaires": 4, "disponible": true}]. 1) Écris la data class Kotlin complète de cet objet. 2) Écris la route Retrofit GET livres/{id} qui renvoie UN tel objet. 3) Le champ JSON s'appelle nbExemplaires mais tu veux la propriété Kotlin « exemplaires » : que faut-il ajouter avec Gson ?

✔ Corrigé
1) data class LivreDto(val id: Long, val titre: String, val auteur: String, val nbExemplaires: Int, val disponible: Boolean). 2) @GET("livres/{id}") suspend fun livre(@Path("id") id: Long): LivreDto. 3) Il faut annoter la propriété pour faire la correspondance explicite : @SerializedName("nbExemplaires") val exemplaires: Int — sans cela, Gson cherche un champ JSON nommé « exemplaires », ne le trouve pas et laisse la valeur par défaut du type, source de bogues silencieux.

Exercice 3

Préparation de publication : l'application BiblioIAM Mobile est en version 1.1.0 (versionCode 5) sur la piste de test fermée. La nouvelle version corrige un plantage et ajoute l'écran « Ma bibliothèque ». a) Quels versionCode et versionName proposer pour l'envoi en production ? b) Liste, dans l'ordre, cinq éléments à vérifier avant l'envoi du bundle. c) Le responsable propose de « profiter de la mise à jour pour ajouter la permission de localisation précise, au cas où » : ta réponse argumentée.

✔ Corrigé
a) versionCode 6 (obligatoirement supérieur à 5) et versionName « 1.2.0 » (ajout de fonction visible = le deuxième chiffre augmente, selon l'usage courant). b) Vérifications : le bundle est signé avec LE keystore d'origine ; le versionCode est incrémenté ; l'application a été testée en build release (pas seulement en debug) sur un appareil propre ; les permissions du manifeste sont toutes justifiées ; la fiche (captures, description) reflète la nouvelle version. c) Refus argumenté : une permission « au cas où » effraie les utilisateurs, expose à un refus de la boutique pour les permissions sensibles non justifiées, et contredit la règle du cours — chaque permission doit correspondre à une fonction réelle et explicable dans la fiche. On l'ajoutera le jour où une fonction de localisation existera, avec son explication.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quel composant Android convient pour continuer un téléchargement après la fermeture de l'écran, sans interface ?

A) Une Activity

B) Un Service

C) Un TextView

2. Au démarrage normal d'un écran, l'ordre exact des rappels est…

A) onCreate, onResume, onStart

B) onStart, onCreate, onResume

C) onCreate, onStart, onResume

3. Les permissions demandées par l'application se déclarent dans…

A) Le fichier strings.xml

B) AndroidManifest.xml

C) Le layout de l'écran

4. Pour chercher un livre dont le titre contient un mot saisi par l'utilisateur, la requête sûre est…

A) "titre LIKE '%" + mot + "%'"

B) Un tri de toute la table en mémoire

C) "titre LIKE ?" avec le mot passé en argument

5. Que se passe-t-il si un appel réseau est exécuté sur le thread principal ?

A) L'appel devient juste plus lent

B) Android lève NetworkOnMainThreadException et l'application plante

C) Le système déplace automatiquement l'appel sur un autre thread

6. Pour publier sur Google Play, le format de paquet attendu est…

A) Le fichier .aab (Android App Bundle)

B) Un dossier de sources compressé en .zip

C) L'APK de débogage généré par Android Studio

7. L'équipe a perdu le keystore de signature de son application déjà publiée. Conséquence ?

A) Aucune : la clé sert seulement au premier envoi

B) La boutique régénère automatiquement la même clé

C) Impossible de publier des mises à jour sous la même identité, sauf procédure de réinitialisation encadrée

8. L'utilisateur tourne son téléphone pendant qu'il lit un écran. Par défaut, l'Activity…

A) Est détruite puis recréée : l'état doit être sauvegardé (ViewModel, onSaveInstanceState)

B) Ne reçoit aucun rappel du système

C) Passe seulement par onPause puis onResume, sans risque

9. Le rôle du RecyclerView (avec son Adapter) est…

A) De signer l'application avant publication

B) D'afficher de longues listes en ne créant que les lignes visibles et en les recyclant

C) De remplacer la base SQLite pour les listes

10. Dans une architecture propre, Retrofit sert à…

A) Dessiner les écrans plus vite

B) Stocker les données hors ligne

C) Transformer une interface Kotlin annotée (@GET, @POST…) en client d'API REST

Corrigés du QCM

1. Réponse B — Le Service exécute des tâches de fond sans interface ; l'Activity est liée à un écran et peut être détruite quand on le quitte.

2. Réponse C — Création, visibilité, premier plan : onCreate → onStart → onResume, sans inversion possible.

3. Réponse B — Le manifeste est la carte d'identité de l'application : composants, permissions et point d'entrée y sont déclarés.

4. Réponse C — Le paramètre lié (?) traite la saisie comme une donnée : impossible de casser la requête ou d'injecter du SQL.

5. Réponse B — Le réseau sur le thread d'interface est interdit : exception et plantage. On passe par une coroutine (Dispatchers.IO).

6. Réponse A — L'AAB permet à Google de générer l'APK optimisé pour chaque appareil ; l'APK de debug n'est pas publiable.

7. Réponse C — La signature est l'identité de l'application : une mise à jour doit porter la même, d'où l'obligation de sauvegarder le keystore en plusieurs lieux sûrs.

8. Réponse A — Un changement de configuration détruit et recrée l'écran : sans sauvegarde d'état, saisies et résultats sont perdus.

9. Réponse B — RecyclerView + Adapter = listes longues et fluides : création limitée aux lignes visibles, recyclage des autres.

10. Réponse C — Retrofit génère les appels HTTP et la conversion JSON à partir d'une simple interface déclarative.

Résumé — les notions importantes

• Une application Android est un assemblage de quatre composants — Activity (écran), Service (fond), BroadcastReceiver (annonces), ContentProvider (données partagées) — déclarés dans AndroidManifest.xml, qui porte aussi les permissions.

• Les interfaces sont des arbres de vues XML reliés au code par leurs id ; RecyclerView et son Adapter affichent les longues listes ; textes et couleurs vivent dans les ressources, jamais en dur dans le code.

• Le cycle de vie d'une Activity suit un ordre fixe — onCreate → onStart → onResume → onPause → onStop → onDestroy (+ onRestart au retour) — et la rotation impose de sauvegarder l'état.

• Stockage local : SharedPreferences pour les réglages clé-valeur, fichiers pour les contenus, SQLite (via SQLiteOpenHelper puis Room) pour les données structurées — avec des requêtes toujours paramétrées.

• Le réseau se consomme en asynchrone : Retrofit + data class pour le JSON, coroutine sur Dispatchers.IO, permission INTERNET, états chargement/erreur et repli hors-ligne.

• Publier = signer avec un keystore précieusement sauvegardé, un versionCode toujours croissant, le format AAB, une fiche complète et des permissions toutes justifiées.

• Un projet mobile réussi se construit en jalons testés — écrans, stockage, réseau, publication — et se démontre sur les cas limites : hors-ligne, rotation, base vide.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Cours complet : Maîtrisez le Développement Android Moderne avec Jetpack Compose | 2025

Chaîne : Malakisi · Langue : Français · Niveau : Débutant

Un cours complet et récent, en français, qui construit des applications Android modernes en Kotlin de la première Activity aux données : parfait pour voir en vrai l'architecture et l'enchaînement des écrans de la leçon. Avance ensuite jusqu'aux parties sur les données pour préparer les leçons 4 et 5.

https://www.youtube.com/watch?v=56l2yPRE32I

Leçon 2 — TP1 Developpement Application Mobile sous Android

Chaîne : Aly FOFANA · Langue : Français · Niveau : Débutant

Un travaux pratique guidé en français : environnement Android Studio, structure d'un projet, création d'un layout et chargement de ce layout depuis une Activity. C'est exactement le geste de la leçon 2 — regarde la vidéo en refaisant chaque étape sur ton propre projet.

https://www.youtube.com/watch?v=z3PKUwA_vIE

Leçon 4 — TUTO Android : Manipulation d'une base de données SQLite via l'API SQLiteOpenHelper

Chaîne : Dominique Liard · Langue : Français · Niveau : Intermédiaire

Le tutoriel de référence de Koor.fr sur SQLiteOpenHelper : création de la base, insertion, lecture et mise à jour des données depuis une application Android. Il montre en code réel tout ce que la leçon 4 explique — compare sa classe d'aide avec notre BiblioDbHelper et repère où passent les paramètres des requêtes.

https://www.youtube.com/watch?v=BX_0RItiwcc

Leçon 5 — API REST JSON - Explication et exemples

Chaîne : Arthur Janssens · Langue : Français · Niveau : Débutant à intermédiaire

Une explication claire, en français, de ce qu'est une API REST et de pourquoi les applications natives — Android compris — s'y connectent pour lire leurs données en JSON. À regarder avant de manipuler Retrofit : tu comprendras ce que le client de la leçon 5 consomme réellement, côté serveur comme côté téléphone.

https://www.youtube.com/watch?v=UQwjytQzoqE

Leçon 6 — Publier son app : build, signature, stores | Développement mobile #8

Chaîne : Bitdyne · Langue : Français · Niveau : Intermédiaire

L'épisode de la formation mobile de Bitdyne consacré à la sortie d'une application : le build, la numérotation des versions, la signature qui prouve l'origine, puis le chemin vers les boutiques Google Play et App Store. Il illustre point par point la leçon 6, avec

un contexte francophone proche du tien.

https://www.youtube.com/watch?v=Qooh442N-V0

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Documentation officielle Android Developers (developer.android.com) : guides sur les composants, le cycle de vie des Activities, le stockage et la publication — la source à consulter en premier, en français pour une grande partie.

• Documentation du langage Kotlin (kotlinlang.org) : coroutines, fonctions suspendues et classes de données utilisées dans tout le cours.

• Documentation de Retrofit (square.github.io/retrofit) : déclaration des interfaces d'API et convertisseurs JSON.

• Aide Google Play Console (support.google.com/googleplay/android-developer) : signature, App Bundle, fiches produit et règles de publication.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !